

Reducción de Falsas Alarmas en la Detección Satelital de Incendios Forestales mediante Análisis Multivariable Acelerado por GPU sobre HPC

Entrega 2: Implementación Inicial y Validación Parcial

Allan Arnulfo Montes Monge, Esteban Gutiérrez Saborío, Maria Moroney Sole,

Marlon Mora Delgado y Robson Sthiffen Calvo Ortega

Curso TTCT0017 – Computación Paralela y Distribuida

LEAD University

Profesor: Johansell Villalobos Cubillo

San José, Costa Rica

Repositorio: https://github.com/Allan1492/Deteccion_Incendios_Forestales_Imag_Satelitales

Abstract—Este documento presenta la implementación inicial y la validación parcial del proyecto de detección de incendios forestales a partir de datos satelitales. Se reporta la adquisición y consolidación de un conjunto real de NASA FIRMS con 52 508 485 registros (1.24 GB en formato Parquet), su preprocesamiento mediante motores de cómputo paralelo, un análisis exploratorio espacio-temporal y, de forma central, la primera evaluación cuantitativa del rendimiento del pipeline. La evaluación compara una línea base estrictamente secuencial contra implementaciones paralelas de memoria compartida y distribuidas, ejecutadas sobre el clúster de alto rendimiento Kabré del Centro Nacional de Alta Tecnología. Se reportan speedup, eficiencia, escalabilidad fuerte y débil, throughput y uso de memoria. Los resultados muestran un speedup global de hasta $45\times$ frente a la línea base secuencial, con un punto de saturación del paralelismo en torno a los 16 hilos —coherente con la ley de Amdahl— y un límite de memoria en el modelo de procesos distribuidos que provoca la terminación de los *workers* a partir de cierto grado de paralelismo.

Index Terms—incendios forestales, teledetección, cómputo paralelo, HPC, speedup, escalabilidad, Polars, Dask, SLURM.

I. INTRODUCCIÓN

La Entrega 1 estableció los fundamentos y el diseño del proyecto: la detección de incendios forestales a partir de datos satelitales masivos, con el objetivo específico de *reducir la tasa de falsas alarmas* combinando variables físicas y contextuales de cada detección. El presente documento reporta la materialización de las tres primeras etapas del pipeline —adquisición de datos, preprocesamiento paralelo y análisis exploratorio— junto con la primera evaluación cuantitativa del rendimiento computacional, que constituye el eje central del curso.

El aporte principal de esta entrega es metodológico: se establece un protocolo de medición que permite atribuir correctamente las ganancias de rendimiento al paralelismo, distinguiéndolas de las ganancias derivadas del cambio de tecnología. Esta distinción, que se detalla en la Sección IV,

TABLE I
CARACTERÍSTICAS DEL CONJUNTO DE DATOS CONSOLIDADO

Atributo	Valor
Fuente	NASA FIRMS
Sensor	VIIRS (Suomi-NPP, NOAA-20, NOAA-21)
Periodo	30 jun 2025 – 30 jun 2026
Registros	52 508 485
Columnas	14
Formato	Apache Parquet (Snappy)
Tamaño	1.24 GB (1 332 485 896 bytes)
Grupos de filas	51
Valores nulos	0 en todas las columnas

resulta indispensable para reportar métricas de eficiencia coherentes.

II. ADQUISICIÓN Y GESTIÓN DE DATOS

A. Conjunto de datos

Se consolidó un conjunto real de puntos de calor procedente del sistema FIRMS de la NASA, con las características de la Tabla I. El volumen supera con holgura el mínimo de 1 GB exigido por el curso.

B. Nota sobre la nomenclatura de las bandas

Las columnas `brightness` y `bright_t31` conservan los nombres del producto MODIS, pero los datos provienen de VIIRS. Corresponden, respectivamente, a las bandas `bright_ti4` (canal I-4, $3.74\ \mu\text{m}$, sensible al fuego) y `bright_ti5` (canal I-5, $11.45\ \mu\text{m}$, temperatura de fondo). Esta equivalencia es funcionalmente válida para el propósito del proyecto, ya que el contraste térmico

$$\Delta T = \text{brightness} - \text{bright_t31} \quad (1)$$

mantiene el mismo significado físico —diferencia entre el canal del fuego y el del entorno— que en el producto MODIS.

TABLE II
CONFIGURACIÓN DE EJECUCIÓN EN EL CLÚSTER KABRÉ

Parámetro	Valor
Clúster	Kabré (CeNAT)
Partición	kura
Nodos disponibles	11
Núcleos por nodo	40
Memoria por nodo	257 GB
Gestor de recursos	SLURM
Nodo de ejecución	kura-2b.cnca
Entorno	Miniforge, Python 3.11

Se deja constancia explícita de esta correspondencia para evitar ambigüedades en la interpretación de los resultados.

C. Verificación de integridad

Durante el desarrollo se detectó que una copia previa del archivo estaba truncada, lo que provocó que el análisis exploratorio se ejecutara sobre datos sintéticos de respaldo sin advertencia explícita. Para evitar la repetición de este problema se incorporó al pipeline una rutina de verificación (`verificar_parquet.py`) que comprueba los bytes mágicos `PAR1` al inicio y al final del archivo, valida los metadatos, realiza una lectura de prueba y confirma la presencia de las columnas requeridas. Esta verificación se ejecuta automáticamente antes de cada trabajo de cómputo.

III. INFRAESTRUCTURA DE CÓMPUTO

Los experimentos se ejecutaron en el clúster **Kabré** del Centro Nacional de Alta Tecnología (CeNAT), que dispone de 2048 núcleos de cómputo, 120 TB de almacenamiento y nodos con aceleradores gráficos. La Tabla II resume la partición utilizada.

El envío de trabajos se realiza mediante `sbatch`, reservando un nodo completo para garantizar que las mediciones no se vean afectadas por la carga de otros usuarios. Las variables `OMP_NUM_THREADS`, `OPENBLAS_NUM_THREADS` y `MKL_NUM_THREADS` se fijan en 1 para impedir que las bibliotecas de álgebra lineal generen hilos adicionales que contaminen la medición del paralelismo.

IV. METODOLOGÍA DE EVALUACIÓN DE RENDIMIENTO

A. Diseño experimental

Se evaluaron cuatro configuraciones de procesamiento, descritas en la Tabla III. La línea base es **pandas restringido a un solo hilo**. Esta elección es deliberada: el motor *in-memory* de Polars es también multihilo, por lo que compararlo contra su motor *streaming* no constituye una comparación entre ejecución secuencial y paralela, sino entre dos estrategias de ejecución igualmente paralelas.

TABLE III
CONFIGURACIONES EVALUADAS

Motor	Rol	Paralelismo
pandas	Línea base T_1	Un hilo (BLAS restringido)
Polars	Memoria compartida	POLARS_MAX_THREADS
Dask	Distribuido	Número de <i>workers</i>
cuDF	GPU (Etapa 4)	Miles de hilos CUDA

B. Operaciones medidas

Se seleccionaron cuatro operaciones representativas del preprocesamiento real del proyecto, no cargas sintéticas: **carga** (lectura y materialización del Parquet), **filtro** (descarte de detecciones de baja confianza), **agregación** (conteo y FRP medio por satélite y mes) y **características** (cálculo de ΔT según la Ec. 1 y derivación de mes y hora). La última operación construye precisamente la variable central del problema de falsas alarmas, de modo que el benchmark mide el pipeline que efectivamente se utilizará.

C. Aislamiento del número de hilos

El número de hilos de Polars se controla mediante la variable de entorno `POLARS_MAX_THREADS`, que solo surte efecto antes de importar la biblioteca. Por ello cada configuración se ejecuta en un **proceso independiente**, invocado por un script controlador. Medir distintos niveles de paralelismo dentro de un mismo proceso —o de un mismo *notebook*— produciría resultados inválidos.

D. Métricas

Se calculan dos medidas de aceleración con significados distintos:

$$S_{\text{global}}(p) = \frac{T_{\text{pandas}}}{T_p} \quad (2)$$

$$S_{\text{paralelo}}(p) = \frac{T_{\text{motor}}(1)}{T_{\text{motor}}(p)} \quad (3)$$

La Ec. 2 cuantifica la ganancia total de cambiar de tecnología y paralelizar. La Ec. 3 compara cada motor consigo mismo y aísla el efecto del paralelismo. La eficiencia se calcula exclusivamente a partir de la segunda:

$$E(p) = \frac{S_{\text{paralelo}}(p)}{p}. \quad (4)$$

Emplear S_{global} en la Ec. 4 produciría eficiencias superiores al 100 %, atribuyendo al paralelismo una ganancia que en realidad proviene del motor de ejecución. Se reportan además el *throughput* (filas/s y MB/s) y la memoria pico residente.

E. Experimentos de escalabilidad

La **escalabilidad fuerte** mantiene fijo el tamaño del problema (52.5 millones de registros) e incrementa $p \in \{1, 2, 4, 8, 16, 32, 40\}$. La **escalabilidad débil** incrementa el tamaño del problema proporcionalmente a p , de modo que la carga por hilo permanece constante; en el caso ideal el

TABLE IV
RESULTADOS PARA LA OPERACIÓN DE AGREGACIÓN (MEDIANA DE TRES REPETICIONES, 52.5 M REGISTROS)

Motor	p	T (s)	S_{glob}	S_{par}	E (%)
pandas	1	11.60	1.00	—	—
Polars	1	2.33	4.97	1.00	100.0
Polars	4	0.613	18.92	3.80	95.1
Polars	8	0.354	32.77	6.59	82.4
Polars	16	0.255	45.46	9.14	57.1
Polars	32	0.275	42.20	8.48	26.5
Polars	40	0.285	40.68	8.18	20.5
Dask	1	61.80	0.19	1.00	100.0
Dask	2	34.08	0.34	1.81	90.7
Dask	4	33.92	0.34	1.82	45.5
Dask	≥ 8	fallo por memoria (<i>workers</i> terminados)			

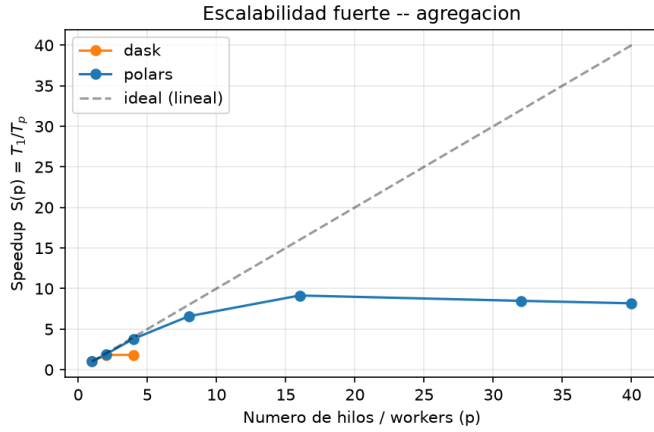


Fig. 1. Escalabilidad fuerte (speedup paralelo) para la operación de agregación. La recta discontinua representa la aceleración lineal ideal; la curva de Polars se aparta de ella y satura en torno a los 16 hilos.

tiempo se mantendría invariante. Cada configuración se repite tres veces y se reporta la **mediana**, más robusta que la media frente a la primera lectura sin caché del sistema de archivos.

V. RESULTADOS PRELIMINARES

A. Tiempos y aceleración

La Tabla IV resume los resultados para la operación de **agregación** (conteo y FRP medio por satélite y mes), que fue la de mayor aceleración. Se reportan el tiempo mediano, ambos speedups y la eficiencia. La descomposición es reveladora: sobre 52.5 millones de registros, el simple cambio de pandas a Polars secuencial ya aporta un factor de $4.97\times$; el paralelismo de Polars añade sobre ese punto un factor adicional de $9.14\times$; y el efecto combinado alcanza un speedup global de $45.46\times$ con 16 hilos.

B. Punto de saturación y ley de Amdahl

El resultado más relevante es que Polars deja de acelerar más allá de los **16 hilos**. En la agregación, el tiempo baja de 2.33 s (1 hilo) a 0.255 s (16 hilos), pero con 32 y 40 hilos vuelve a subir levemente (0.275 s y 0.285 s). En consecuencia,

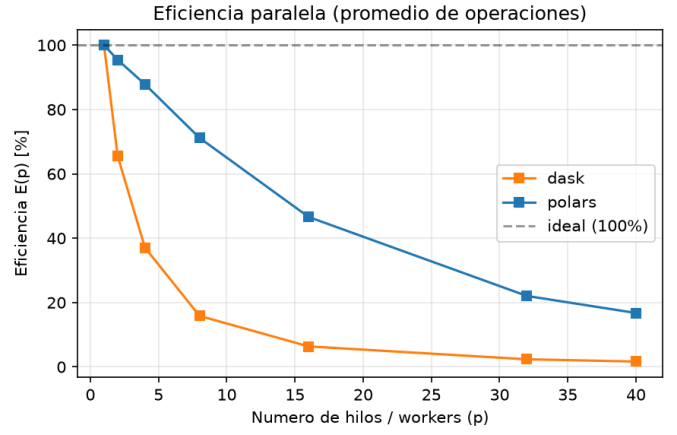


Fig. 2. Eficiencia paralela en función del número de hilos. La caída sostenida al aumentar p refleja el efecto descrito por la ley de Amdahl.

la eficiencia cae de forma sostenida: 95.1 % con 4 hilos, 57.1 % con 16 y apenas 20.5 % con 40. Este comportamiento es consistente con la ley de Amdahl [6]: la fracción no paralelizable de la tarea —lectura del Parquet desde disco, descompresión y coordinación entre hilos— impone un techo a la aceleración, y a partir de cierto punto añadir hilos solo incrementa la contención por el ancho de banda de memoria sin reducir el tiempo. Las cuatro operaciones exhiben el mismo patrón, con el punto de saturación situado entre 8 y 16 hilos.

C. Comparación entre motores

La comparación Polars–Dask es ilustrativa de dos filosofías de paralelismo. Polars (hilos sobre memoria compartida) es consistentemente el más rápido en este escenario de un solo nodo. Dask (procesos distribuidos) parte de una desventaja notable —su agregación con un *worker* tarda 61.8 s frente a 2.33 s de Polars— debido a la sobrecarga de serialización y coordinación entre procesos; aunque escala casi linealmente al pasar a 2 *workers* (34.1 s), no alcanza a Polars. Este resultado no descalifica a Dask: su modelo está diseñado para conjuntos que exceden la memoria de una máquina o para ejecución multinodo, escenario que se explorará en la Entrega 3.

D. Límite de memoria en el paralelismo por procesos

Un hallazgo adicional, y didácticamente valioso, es que las operaciones de *agregación* y *filtrado* con Dask **fallaron** al emplear 8 o más *workers*, con el error “*all those workers died while running it*”. La causa es la saturación de memoria: en el modelo de Dask cada *worker* es un proceso independiente que mantiene su propia porción de datos, de modo que el consumo total crece con el número de *workers* hasta agotar la memoria del nodo, momento en el que el planificador termina los procesos. Esto evidencia una limitación intrínseca del paralelismo por réplica de memoria, ausente en el modelo de hilos de Polars, que comparte un único espacio de direcciones. La operación de *características*, más ligera, sí completó en todas las configuraciones.

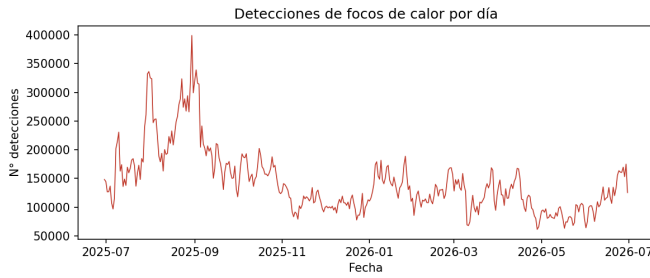


Fig. 3. Serie temporal diaria de detecciones. El pico de finales de agosto corresponde a la temporada de incendios boreal.

E. Escalabilidad débil

En el experimento de escalabilidad débil, donde el volumen crece en proporción al número de hilos, el *throughput* de Polars se mantiene elevado y relativamente estable (por ejemplo, en la operación de características pasa de 21.6 a 133.8 millones de filas/s al escalar de 1 a 40 hilos), lo que indica que el sistema absorbe cargas mayores sin degradación proporcional del tiempo mientras la memoria lo permita.

VI. ANÁLISIS EXPLORATORIO

El análisis exploratorio se ejecutó sobre los 52.5 millones de focos de calor reales, empleando el mismo motor *streaming* de Polars para procesar el conjunto sin materializarlo por completo en memoria.

A. Distribución temporal y espacial

La serie temporal diaria (Fig. 3) revela una marcada estacionalidad: la actividad alcanza su máximo a finales de agosto de 2025 (hasta 399 153 detecciones en un solo día, el 30 de agosto) y su mínimo durante el invierno boreal. Espacialmente, las mayores densidades se concentran en el centro de Canadá (celda 56°N, 100°O), Siberia y el África subsahariana. Los tres satélites VIIRS (NOAA-20, Suomi-NPP y NOAA-21) aportan volúmenes casi idénticos (en torno a 17 millones de detecciones cada uno), lo que indica una cobertura temporal homogénea.

B. Variable objetivo

Un hallazgo con implicaciones directas para el modelado es la distribución de la variable objetivo. La Tabla V muestra que las detecciones de baja confianza —que en la formulación del problema operan como *proxy* de falsa alarma— representan el 12.15 % del total.

Esta distribución implica un desbalance aproximado de 7:1 entre clases. En consecuencia, la exactitud (*accuracy*) no es una métrica adecuada: un clasificador que asignara la clase mayoritaria a todos los registros alcanzaría un 87.85 % sin capacidad discriminativa alguna. La evaluación del modelo se basará por tanto en precisión, exhaustividad (*recall*) y matriz de confusión, tal como se estableció en la Entrega 1.

TABLE V
DISTRIBUCIÓN DEL NIVEL DE CONFIANZA DE LAS DETECCIONES

Valor	Nivel	Registros	%
n	Nominal	43 214 217	82.30
l	Bajo	6 382 160	12.15
h	Alto	2 912 108	5.55
Total		52 508 485	100.00

TABLE VI
CONTRASTE TÉRMICO Y POTENCIA RADIATIVA MEDIOS POR NIVEL DE CONFIANZA

Confianza	ΔT medio (K)	FRP medio (MW)	Registros
Alta (h)	60.81	28.21	2 912 108
Nominal (n)	34.78	8.41	43 214 217
Baja (l)	28.11	12.79	6 382 160

C. Validación empírica de la hipótesis

El análisis aporta evidencia directa a favor de la hipótesis planteada en la Entrega 1, según la cual el contraste térmico $\Delta T = \text{brightness} - \text{bright_t31}$ discrimina las detecciones confiables de las probables falsas alarmas. La Tabla VI muestra el ΔT medio y la potencia radiativa media por nivel de confianza: las detecciones de alta confianza presentan un contraste térmico de 60.8 K, más del doble que las de baja confianza (28.1 K). Es decir, los focos con menor diferencia de temperatura respecto a su entorno —los que apenas destacan del fondo— tienden a clasificarse con baja confianza, exactamente el comportamiento que se anticipó. Este resultado confirma que las variables físicas contienen señal discriminativa suficiente para el modelo de la Etapa 4.

VII. LIMITACIONES Y TRABAJO PENDIENTE

A. Limitaciones de esta entrega

- Las mediciones corresponden a un único nodo (paralelismo de memoria compartida). La evaluación multinodo con Dask distribuido queda pendiente.
- La rama de imágenes satelitales del conjunto de Kaggle aún no ha sido incorporada al pipeline.
- No se ha ejecutado la evaluación con aceleración por GPU, prevista para la Etapa 4 en las particiones `nukwa-140s` del clúster.

B. Trabajo pendiente

Para la Entrega 3 se contempla: el entrenamiento del clasificador tabular con RAPIDS cuML sobre la variable objetivo definida; la implementación de la red neuronal convolucional en PyTorch sobre el conjunto de imágenes; la evaluación de rendimiento con GPU y en configuración multinodo; y la construcción del *dashboard* interactivo con Plotly y Dash.

VIII. REPRODUCIBILIDAD

El código y los datos están disponibles en el repositorio del proyecto. Los pasos mínimos para reproducir los resultados de esta entrega son:

- 1) Montar el entorno en el clúster: `bash entrega2_benchmark/setup_kabre.sh`
(instala Python 3.11, Polars, Dask y dependencias).
- 2) Verificar la integridad del Parquet: `python3 verificar_parquet.py <ruta>.parquet`.
- 3) Lanzar el *benchmark* en Kabré: `sbatch entrega2_benchmark/submit_kabre.slurm`.
- 4) Generar tablas y figuras: `python3 analizar_resultados.py <resultados>.csv -figuras figuras/`.
- 5) Ejecutar el análisis exploratorio: `python3 eda/eda_real.py <ruta>.parquet eda_salida`.

El *notebook* de preprocesamiento y las instrucciones detalladas se encuentran en el README del repositorio.

IX. CONCLUSIÓN

Esta entrega consolidó un conjunto de datos real de 52.5 millones de registros verificado en su integridad, estableció un entorno reproducible de cómputo en el clúster Kabré y definió un protocolo de medición de rendimiento metodológicamente riguroso, que distingue la ganancia atribuible al paralelismo de la derivada del motor de ejecución. Los resultados confirman aceleraciones globales de hasta $45\times$, identifican el punto de saturación del paralelismo de memoria compartida en torno a los 16 hilos —de acuerdo con la ley de Amdahl— y evidencian el límite de memoria del paralelismo por procesos. Estos hallazgos validan el diseño propuesto en la Entrega 1 y establecen la línea base sobre la que se evaluarán, en la Entrega 3, las etapas de modelado, la aceleración por GPU y la ejecución multinodo.

REFERENCES

- [1] W. Schroeder, P. Oliva, L. Giglio, y I. A. Csiszar, “The New VIIRS 375 m active fire detection data product: Algorithm description and initial assessment,” *Remote Sensing of Environment*, vol. 143, pp. 85–96, 2014.
- [2] L. Giglio, W. Schroeder, y C. O. Justice, “The collection 6 MODIS active fire detection algorithm and fire products,” *Remote Sensing of Environment*, vol. 178, pp. 31–41, 2016.
- [3] NASA, “Fire Information for Resource Management System (FIRMS).” [En línea]. Disponible: <https://firms.modaps.eosdis.nasa.gov/>
- [4] M. Rocklin, “Dask: Parallel computation with blocked algorithms and task scheduling,” en *Proc. 14th Python in Science Conf. (SciPy)*, 2015, pp. 130–136.
- [5] R. Vink y col., “Polars: Lightning-fast DataFrame library for Rust and Python.” [En línea]. Disponible: <https://pola.rs/>
- [6] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” en *Proc. AFIPS Spring Joint Computer Conf.*, 1967, pp. 483–485.
- [7] J. L. Gustafson, “Reevaluating Amdahl’s law,” *Communications of the ACM*, vol. 31, no. 5, pp. 532–533, 1988.
- [8] Colaboratorio Nacional de Computación Avanzada, “Kabré: infraestructura de supercomputación del CeNAT.” [En línea]. Disponible: <https://kabre.cenat.ac.cr/>
- [9] A. B. Yoo, M. A. Jette, y M. Grondona, “SLURM: Simple Linux Utility for Resource Management,” en *Job Scheduling Strategies for Parallel Processing*, 2003, pp. 44–60.